MLOps compact cheatsheet

STEP 1 Set up and plan

1.1 Set up your computer for machine learning

The toolbox Python 3.12 or newer, terminal, .venv per project, pip or uv, Git, VS Code, later Docker and a GPU or Colab.

PATH The first match in PATH runs. activate puts .venv first. python -m pip keeps pip and python together.

Per system macOS and Linux: python3, .venv/bin. Windows: python or py, .venv\Scripts, execution policy. ChromeOS: Linux container, Debian steps.

Reproducible pip freeze > requirements.txt, pip install -r requirements.txt. Commit requirements, ignore .venv. Never sudo pip.

Docker Desktop on macOS and Windows (WSL 2), Engine on Linux, docker group, docker run hello-world.

GPU NVIDIA: driver, nvidia-smi, CUDA build of PyTorch. Apple silicon: MPS. Chromebook: Colab or Kaggle.

EXAM Two names, one idea: macOS, Linux and ChromeOS call it `python3`, Windows calls it `python` or `py`. Inside an activated virtual environment, `python` works everywhere.

EXAM The program shows what python -m venv creates: a bin folder (Scripts on Windows), include and lib folders, and pyvenv.cfg. The new python reports that it is inside a virtual environment, its packages go to the environment's own site-packages, and include-system-site-packages = false means it cannot see packages installed for the whole computer.

1.2 The MLOps lifecycle and reproducible projects

MLOps Version, test, deploy, monitor and retrain ML. DevOps plus data and models.

Levels 0 manual, 1 automated training pipeline (continuous training), 2 CI/CD for the pipeline.

Reproducible Fix seeds, report spread over seeds, save run records: data fingerprint, params, metrics, versions.

Versions Code in Git, data by hash (DVC), models in a registry, environment in requirements or images. Lineage.

Tests Data, features, model quality and behaviour, infrastructure. Notebooks to tested functions.

Why it matters Models decay as data changes. Hidden technical debt lives outside the model code.

An ML system
behaviour = code + data + model + configuration

Continuous training
new data → retrain → evaluate → register → deploy

EXAM The first block changes every time you run it, which is the point. With the seed fixed at 42, five runs give exactly the same accuracy, 0.972, five times. Over ten different seeds the accuracy has mean 0.956 and standard deviation 0.014, ranging from 0.937 to 0.972, so a difference of 0.01 between two models trained once means nothing.

EXAM The original breast cancer data gives a 5 fold accuracy of 0.981. Flipping 40 labels (7 percent) drops it to 0.882 with the same number of rows. Losing about 30 percent of the rows (399 left) gives 0.975. Each version has a different fingerprint, so the run record shows exactly which one a model saw.

STEP 2 Build and ship

2.1 Pipelines and saving models

Pipeline Preprocessing plus model in one object. ColumnTransformer: numbers impute and scale, categories one hot.

fit and transform Steps learn on training rows only, reuse at predict.
handle_unknown='ignore' for unseen categories.

Leakage No step may see test rows. Selection outside CV: 0.85 on noise, inside: 0.48. Split by time and group too.

Saving joblib.dump the whole pipeline plus JSON metadata (features, classes, versions). Load trusted files only.

Formats joblib for own Python services, skops for safe sharing, ONNX for other languages.

Input contract Check presence, type, range, unknown categories, extra fields. Reject with 422.

Standard scaling $z = (x - \mu_{train}) / \sigma_{train}$

Median imputation
missing x → median of training x

One hot columns
one column per category seen in training

EXAM The churn pipeline reaches a 5 fold ROC AUC of 0.751 and a test accuracy of 0.69. After preprocessing, 5 raw columns have become 9 (3 scaled numbers and 6 one hot columns, with names such as cat__plan_basic). A new customer with a missing bill and a plan called student, which the model never saw, still gets a churn probability of 0.782 instead of an error.

EXAM On 100 rows of random noise with random labels, choosing the 20 best of 5000 columns before cross validation reports an accuracy of 0.85. Doing the same selection inside the pipeline reports 0.48, the honest answer for labels that are pure chance.

EXAM The fitted iris pipeline takes about 1.5 KB on disk. After loading, its predictions match the original on every row, and the flower [5.9, 3.0, 5.1, 1.8] is predicted as virginica. The JSON file records scikit-learn 1.8.0, which the serving side should match.

2.2 Serving models with Flask, FastAPI and Django

Model API Load once. /health plus POST /predict. JSON in and out. 200 success, 422 bad content, 405 wrong method, 500 is a bug.

Frameworks Flask small. FastAPI checks Pydantic classes, docs at /docs. Django for a model inside a full web app.

Test then call Test client in one process. Then curl (macOS, Linux, ChromeOS) or Invoke-RestMethod and curl.exe (Windows).

Online or batch Online answers during the visit. Batch scores on a schedule, cheaper, less fresh. Read big files in chunks.

Production servers Gunicorn on macOS and Linux, Waitress on Windows, Uvicorn for FastAPI everywhere. Several workers. Never app.run in production.

Capacity L = λW. Workers = λ × latency / target use. M/M/1: W = 1/(μ − λ) explodes near capacity.

Latency
time from request sent to response received

Throughput of one worker
requests per second = 1000 / latency in ms

Little's law $L = \lambda \times W$

Workers needed
workers = λ × latency / target use

EXAM One bad request produced three precise errors at once: sepal_length must be greater than 0, sepal_width is not a number, and petal_width is missing. The batch route scored two flowers in a single predict_proba call, virginica at 0.703 and setosa at 0.981.

TRAP Django gives the same answers as Flask (200 with virginica at 0.703, 422 for bad JSON), and @require_POST turns a GET into 405 without running the view. In a real project the view lives in an app inside `django-admin startproject`, and the settings come from environment variables, never a SECRET_KEY written in the code.

EXAM A queue does not grow in a straight line. One worker that can serve 50 requests per second answers in 25 ms at 10 per second, 100 ms at 40 per second, and 1 full second at 49 per second. That is why services keep spare capacity.